物理学セミナー：非線形・非平衡物理学入門
第 2 回：微分方程式の数値解法

川﨑猛史・吉野元

大阪大学 D3 センター・大学院理学研究科 物理学専攻

最終更新日: April 23, 2026

第 2 回講義資料目次

1 本セミナーのスケジュール

2 微分方程式の数値解法
- 離散化: Euler 法
- 無次元化

3 数値計算
- Python 環境の構築
- 数値計算プログラム

4 第 2 回のまとめ

2.1. 本セミナーのスケジュール

1. 4/16：第 1 回
2. 4/23：第 2 回
3. 4/30：休講（銀杏祭準備）
4. 5/7：第 3 回
5. 5/14：第 4 回
6. 5/21：第 5 回
7. 5/28：第 6 回
8. 6/4：第 7 回
9. 6/11：第 8 回

第 2 回講義資料目次

1 本セミナーのスケジュール

2 微分方程式の数値解法
- 離散化: Euler 法
- 無次元化

3 数値計算
- Python 環境の構築
- 数値計算プログラム

4 第 2 回のまとめ

2.2.1. 離散化: Euler 法

♣ 離散化: **Euler** 法

次に本節では，微積分の数値計算において必要になる離散化（差分化）の方法について説明する．

- 前回と同様に，以下の通り抵抗の働く質点の運動方程式

$$m\frac{\mathrm{d}v(t)}{\mathrm{d}t} = -\zeta v(t) \tag{1}$$

  を考える．
- もちろんこの問題は解析的に解けるが，今回はこれを数値計算解くことにする。
- 数値計算では，連続的な関数方程式を，離散化し，数値積分することにより解く．
- 数値積分の精度を上げることは，問題によっては非常に重要になるが，まずは精度が低いが非常に簡便である Euler 法を紹介する．

2.2.1. 離散化: Euler 法 (2)

- まず，関数 $f(t)$ の時間に関する一階微分は厳密に

$$\frac{\mathrm{d}f(t)}{\mathrm{d}t} = \lim_{\Delta t \to 0} \frac{f(t+\Delta t) - f(t)}{\Delta t} \tag{2}$$

  である．

- 一方，数値計算では、$\Delta t \to 0$ の極限を厳密にとることはできず，各時間ステップで Δt にできるだけ小さな値を当てはめ，以下のように近似計算を行う．特に，以下方法を $\boxed{\text{Euler 法}}$ と呼ぶ.

$$\boxed{\frac{\mathrm{d}f(t)}{\mathrm{d}t} = \frac{f(t+\Delta t) - f(t)}{\Delta t} + \mathcal{O}((\Delta t))} \tag{3}$$

- Euler 法は厳密な微分と比較して Δt のオーダーの誤差を累積するため，問題によっては機能しない離散手法である．
- 一方，今回扱う問題では，保存系ではないので，ある程度の精度を得ることができる．

2.2.1. 離散化: Euler 法 (3)

- Euler 法を用いることで式 (1) は

$$m\frac{v(t+\Delta t)-v(t)}{\Delta t}=-\zeta v(t). \tag{4}$$

と離散化することができ，運動方程式は

$$v(t+\Delta t)=v(t)-\frac{\zeta v(t)\Delta t}{m} \tag{5}$$

となる．

2.2.2. 無次元化

♣ 無次元化

ここでは，物理変数（定数）の無次元化について扱う．

- 計算機が扱うことが出来る数は基本的に無次元数である．
- そのため，物理次元をもつ方程式を計算機で計算させるためには，物理的単位と無次元数に分ける必要がある．
- そこで，前節で扱った微分方程式を計算機で解くために，無次元化する．
- ここでは，まず時間を $t = t_0\tilde{t}$，速度 $v = v_0\tilde{v} = \frac{a}{t_0}\tilde{v}$ と置こう．
- すると，式 (5) は，

$$
\begin{aligned}
\frac{a}{t_0}\tilde{v}(\tilde{t}+\tilde{\Delta}t) &= \frac{a}{t_0}\tilde{v}(\tilde{t}) - \frac{\zeta a \Delta\tilde{t}}{m}\tilde{v}(\tilde{t}) \\
\tilde{v}(\tilde{t}+\tilde{\Delta}t) &= \tilde{v}(\tilde{t}) - \frac{\zeta t_0}{m}\tilde{v}(\tilde{t})\Delta\tilde{t}
\end{aligned}
\tag{6}
$$

となる．

2.2.2. 無次元化 (2)

- ここで，$\frac{\zeta t_0}{m}$ は無次元パラメータであり，基本的にどんな数字を当てはめることも原理的には可能であるが，ここでは簡単のために 1 と置こう．この様にするためには，時間の単位を

$$\boxed{t_0 = \frac{m}{\zeta}} \tag{7}$$

と選んであげればよいことが分かる．この時間スケールは，速度減衰の緩和時間となる！

- ここで，$t_0 = \frac{m}{\zeta}$ つまり，$\frac{\zeta t_0}{m} = 1$ として選べば，差分方程式は

$$\boxed{\tilde{v}(\tilde{t} + \Delta\tilde{t}) = \tilde{v}(\tilde{t}) - \tilde{v}(\tilde{t})\Delta\tilde{t}} \tag{8}$$

という極めて単純なものとなる．

- これを数値的に解くことは，$\tilde{v}(\tilde{t})$ に関する漸化式を解くことである．

第 2 回講義資料目次

1 本セミナーのスケジュール

2 微分方程式の数値解法
- 離散化: Euler 法
- 無次元化

3 数値計算
- Python 環境の構築
- 数値計算プログラム

4 第 2 回のまとめ

2.3.1. Python 環境の構築

♣ Python 環境の構築

- 本実習では Python のプログラムを Jupyter Notebook を用いて実行する．
- Jupyter Notebook は Anaconda Navigator（[本家リンク] 図 **1**）の中に格納されている．
- Anaconda Navigator のインストール方法は以下を参照せよ．
  - **Windows/mac:** Anaconda Navigator のインストール [**windows** マニュアル] [**mac** マニュアル]
- Jupyter note book のインストール方法は以下の図 **2** に示した．

2.3.1. Python 環境の構築 (2)

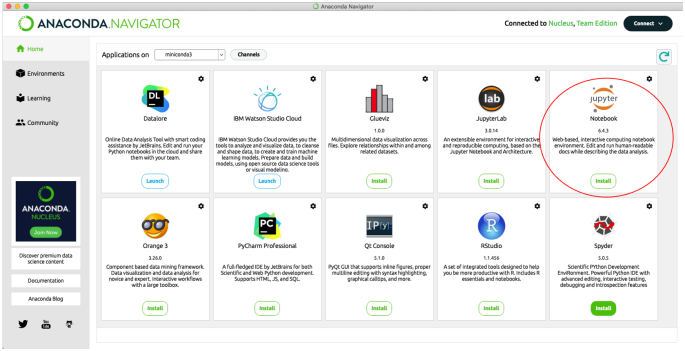

図 1: Anaconda Navigator. この中にある Jupyter note book(赤丸) をインストールし実行．これで Python が使えるようになる．

2.3.1. Python 環境の構築 (3)

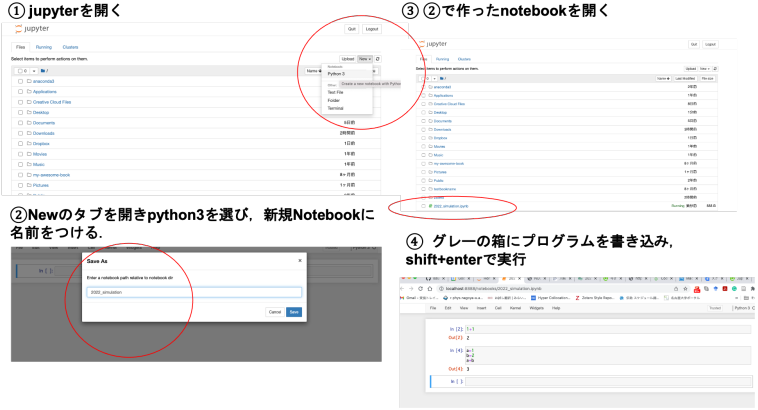

図 2: Jupyter note book 起動後から python 実行まで．

2.3.2. 数値計算プログラム

♣ 数値計算プログラム

- 運動方程式

$$m\dot{v} = -\zeta v \tag{9}$$

の無次元化差分方程式

$$\tilde{v}(\tilde{t} + \Delta\tilde{t}) = \tilde{v}(\tilde{t}) - \tilde{v}(\tilde{t})\Delta\tilde{t} \tag{10}$$

を解く，Python を用いたプログラムを以下に載せる．

- 特に，時間の離散化に関しては $\Delta\tilde{t} = 0.01$ とした．
- この理論解は

$$v(t) = v(0)e^{-\frac{\zeta}{m}t} \tag{11}$$

である．時間の単位を $t_0 = \frac{m}{\zeta}$，また，速度の初期条件を $v(0) = a/t_0$ などとおき，この理論解を無次元化すると，

$$\tilde{v}(\tilde{t}) = e^{-\tilde{t}} \tag{12}$$

となることに注意せよ．

2.3.2. 数値計算プログラム (2)

Listing 1: damp.py: 数値計算プログラム

```
import numpy as np

dt = 0.01
nstep = 500

v = np.zeros(nstep + 1)
t = np.zeros(nstep + 1)
v[0] = 1.0

for i in range(nstep):
    v[i+1] = v[i] - v[i] * dt
    t[i+1] = t[i] + dt

for i in range(nstep):
    print("{:.3g} {:.10g}".format(t[i], v[i]))
```

- さらに，数値計算に加えて図示するプログラムを以下に示す．
- 図示は Python の matplotlib を用いる．

2.3.2. 数値計算プログラム (3)

Listing 2: damp_plot.py: 数値計算および図示

```
import matplotlib
import matplotlib.pyplot as plt
#%matplotlib inline
%config InlineBackend.figure_format = 'retina'

import numpy as np

# フォントTex
plt.rcParams["text.usetex"] = True

# ---- データ生成 ----
dt = 0.01
nstep = 500

v = np.zeros(nstep + 1)
t = np.zeros(nstep + 1)

v[0] = 1.0

for i in range(nstep):
    v[i+1] = v[i] - v[i] * dt
    t[i+1] = t[i] + dt
```

2.3.2. 数値計算プログラム (4)

```

# ---- プロット ----
fig = plt.figure(figsize=(7,7))
ax = fig.add_subplot(111)

ax.set_xlim(0, t[-1])
ax.set_ylim(0, 1.0)

plt.plot(t, v, "o", linewidth=2, color="b", label=r"$dv/dt = -v$")

# ---- 理論解 ----
v_exact = np.exp(-t)
plt.plot(t, v_exact, "-", linewidth=3, color="r", label=r"$v(t)=e^{-t}$")

# ---- 図の設定 ----
plt.tick_params(which='major', width=1, length=10)
plt.tick_params(which='minor', width=1, length=5)
plt.minorticks_on()
ax.spines['top'].set_linewidth(3)
ax.spines['bottom'].set_linewidth(3)
ax.spines['left'].set_linewidth(3)
ax.spines['right'].set_linewidth(3)

```

2.3.2. 数値計算プログラム (5)

```

# ---- 軸のラベル ----
plt.xlabel(r"$t$", color='k', size=30)
plt.ylabel(r"$v(t)$", color='k', size=30)

plt.xticks(color='k', size=25)
plt.yticks(color='k', size=25)

plt.legend(ncol=1, loc=1, borderaxespad=0, fontsize=25, frameon=True)
plt.subplots_adjust(wspace=0.0, hspace=0.25)


plt.tight_layout()
# ---- 保存（同一ディレクトリ） ----
plt.savefig('v_t.png')
plt.savefig('v_t.pdf')
```

このプログラムを実行すると，ipynb ファイルと同じディレクトリに，以下の図が生成される．

2.3.2. 数値計算プログラム (6)

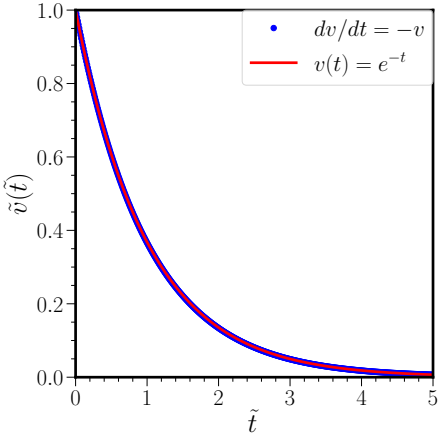

図 3: 結果の出力図.

2.3.2. 数値計算プログラム (7)

第 2 回講義資料目次

1 本セミナーのスケジュール

2 微分方程式の数値解法
- 離散化: Euler 法
- 無次元化

3 数値計算
- Python 環境の構築
- 数値計算プログラム

4 第 2 回のまとめ

2.4. 第 2 回のまとめ

♣ まとめ

- 方程式の無次元化を行った.
- 数値計算環境を構築した.
- 線形 1 階微分方程式の数値計算を行った.

♣ 次回予告

- より高度な微分方程式を解く（理論）.